

National University of Computer and Emerging Sciences



Lab Exercise 08 **DL2001-Introduction to Data Science Lab**

Department of Data Science
FAST-NU, Lahore, Pakistan

Exercise

Use the attached Iris dataset to complete the following tasks

1. k-NN from scratch

- Add the following functions to k-NN class implementation provided in the attached .ipynb file:

- `manhattan_distance(self, x1, x2)`

- `euclidean_distance(self, x1, x2)`

- Complete `_predict` function to implement the following:

- Sort by ascendingly distance and return indices of the given n neighbours

- Get the most frequent class in the array

This should complete your k-NN class implementation. Use this implementation to evaluate your model on this given dataset when:

- `k=7` and `distance_metric='euclidean'`
- `k=7` and `distance_metric='manhattan'`

Based on your result, provide an analysis, which distance is most suitable to use in this scenario.

2. k-NN using built in libraries

- Use the code provided in the attached lab manual to implement k-NN using built in libraries on the given dataset.
- Evaluate your model for the evaluation measures provided in lab manual (Section 3.7) for the following values of k:

- `k=3`

- `k=5`

- `k=7`

Based on your result, provide an analysis, how changing the value of k, affects your results.

3. To visualize the working of k-NN, add the following function to your k-NN from scratch implementation:

```
def visualize(self, x):
```

```

    """
    Visualize the training data, the query point x, and the k
    nearest neighbors.

    Parameters:
    -----
    x : numpy.ndarray, shape (n_features,)
        The query point to classify and visualize (must be 2D
    features).
    """
    if self.X_train.shape[1] != 2:
        raise ValueError("Visualization only supported for 2D
    data.")

    distances = np.array([self.euclidean_distance(x, x_train) for
    x_train in self.X_train])
    n_neighbors_idx = np.argsort(distances)[: self.n_neighbors]

    # Plot training points colored by class
    plt.figure(figsize=(8,6))
    for class_value in np.unique(self.y_train):
        idxs = np.where(self.y_train == class_value)
        plt.scatter(self.X_train[idxs, 0], self.X_train[idxs, 1],
    label=f'Class {class_value}', alpha=0.6)

    # Plot the k nearest neighbors with a circle around them
    neighbors = self.X_train[n_neighbors_idx]
    plt.scatter(neighbors[:, 0], neighbors[:, 1], s=150,
    facecolors='none', edgecolors='k', label='Neighbors')

    # Plot the query point colored by predicted class
    predicted_class = self._predict(x)
    plt.scatter(x[0], x[1], c='red', marker='*', s=200,
    label=f'Query point (Predicted: {predicted_class})')

    plt.title(f'k-NN Visualization (k={self.n_neighbors})')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.legend()
    plt.grid(True)
    plt.show()

```

